

SSAFY

Django

DRF 2

Python



목 차

DRF with N:1 Relation

- 사전 준비
- GET method
- POST method
- 읽기 전용 필드
- DELETE & PUT method
- 읽기 전용 필드 주의 사항

역참조 데이터 구성

- 단일 게시글 + 댓글 목록
- 단일 게시글 + 댓글 개수
- SerializerMethodField
- 역참조 데이터 구성 참고

목 차

API 문서화

- OpenAPI Specification
- 문서화 활용

참고

- 올바르게 404 응답하기
- View와 Serializer의 역할
- DRF 학습 이유

배달 앱에서 여러 사용자가 여러 음식점을 즐겨찾기 등록한다고 생각해 봅시다!

- 예를 들어, 사용자 A는 치킨집과 피자집을, 사용자 B는 피자집과 분식집을 등록할 수 있어요.

1. 사용자와 음식점은 M:N(다대다) 관계를 가집니다.
2. UserFavoriteStore와 같은 [블록] 을 만들어 관계를 정의하고,
3. DRF를 통해 즐겨찾기 목록 조회, 새 즐겨찾기 추가 기능을 구현합니다.
4. [블록] 를 활용해, 클라이언트가 직접 바꾸면 안 되는 값(예: 등록 시간)을 보호할 수 있어요.
5. [블록] 로 가공된 데이터(예: 총 즐겨찾기 수)를 함께 제공할 수도 있습니다

- DRF는 모델 간 복잡한 관계도 쉽게 표현할 수 있게 도와줍니다.
- 중간 테이블을 통한 M:N 구조도 깔끔하게 직렬화 할 수 있고, 읽기 전용 필드와 커스텀 필드를 통해 API의 안정성과 표현력을 높일 수 있습니다.

학습 목표

- ✓ DRF에서 N:1 관계를 표현하는 방법을 이해한다.
- ✓ 외래 키(ForeignKey)를 사용하는 모델 구조를 설계할 수 있다.
- ✓ 댓글 데이터를 조회(GET), 생성(POST), 수정(PUT), 삭제(DELETE)하는 API를 구현할 수 있다.
- ✓ `read_only_fields`를 활용해 클라이언트의 수정이 불가능한 필드를 설정할 수 있다.
- ✓ `SerializerMethodField`를 사용해 응답에 추가적인 정보를 포함하는 방법을 익힌다.
- ✓ 역참조를 통해 부모 객체에서 자식 객체 목록 또는 개수를 응답에 포함하는 방법을 설명할 수 있다.
- ✓ `annotate`를 사용해 모델 데이터를 집계하고, `serializer`에서 이를 활용할 수 있다.
- ✓ `get_object_or_404`, `get_list_or_404`를 활용해 예외 처리를 구현할 수 있다.

DRF with N:1 Relation

사전 준비

사전 준비 (1/3)

- Comment 모델 클래스 정의

```
# articles/models.py
from django.db import models

class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```


사전 준비 (2/3)

- Articles app에 정의된 모델 정보 makemigration

```
$ python manage.py makemigrations
```

Migrations for 'articles':

articles\migrations\0001_initial.py

+ Create model Article

+ Create model Comment

※ 각 명령어 실행 후, 실행 결과 확인 필수!

사전 준비 (3/3)

- 데이터베이스 초기화

```
$ python manage.py migrate
```

Operations to perform:

Apply all migrations: admin, articles, auth, contenttypes, sessions

Running migrations:

Applying contenttypes.0001_initial... OK

- fixtures 데이터 삽입

```
$ python manage.py loaddata articles.json comments.json
```

```
Installed 40 object(s) from 2 fixture(s)
```

※ 각 명령어 실행 후, 실행 결과 확인 필수!

URL 및 HTTP request method 구성

※ 오늘 수업에서 작성할 request method 구성

URL	GET	POST	PUT	DELETE
comments/	댓글 목록 조회			
comments/1/	단일 댓글 조회		단일 댓글 수정	단일 댓글 삭제
articles/1/comments/		댓글 생성		

<표1_Django_DRF 2_request method 구성>

GET method

GET - List (1/3)

- 댓글 목록 조회를 위한 CommentSerializer 정의

```
# articles/serializers.py
from .models import Article, Comment

class CommentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Comment
        fields = '__all__'
```

- url 작성

```
# articles/urls.py
urlpatterns = [
    ...,
    path('comments/', views.comment_list),
]
```



ModelSerializer

Django 모델 구조를 바탕으로
자동으로 필드를 생성해주는
Serializer 클래스

<개념1_Django_DRF 2_ModelSerializer>

GET - List (2/3)

- view 함수 작성

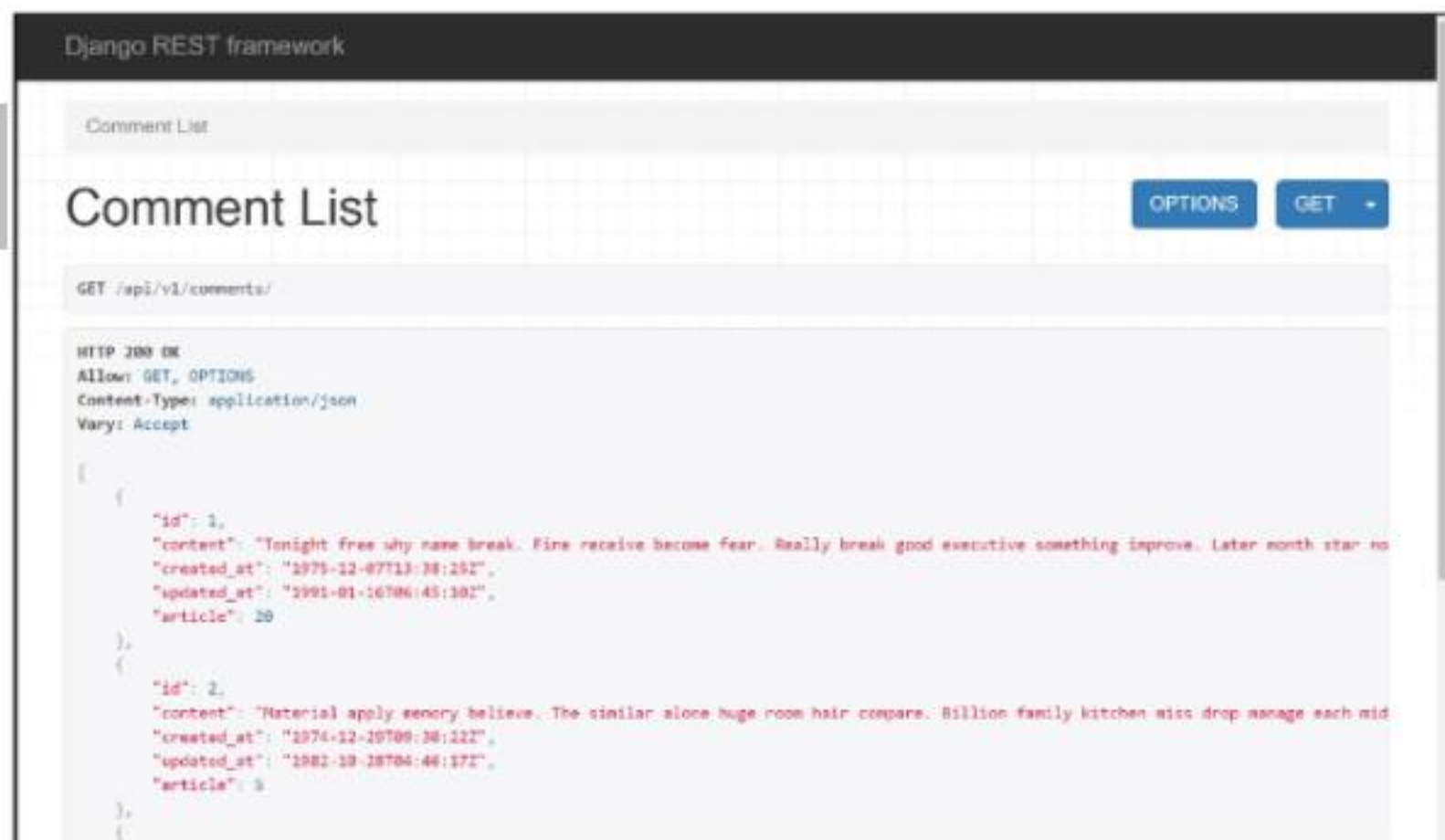
```
# articles/views.py

from .models import Article, Comment
from .serializers import ArticleListSerializer, ArticleSerializer, CommentSerializer

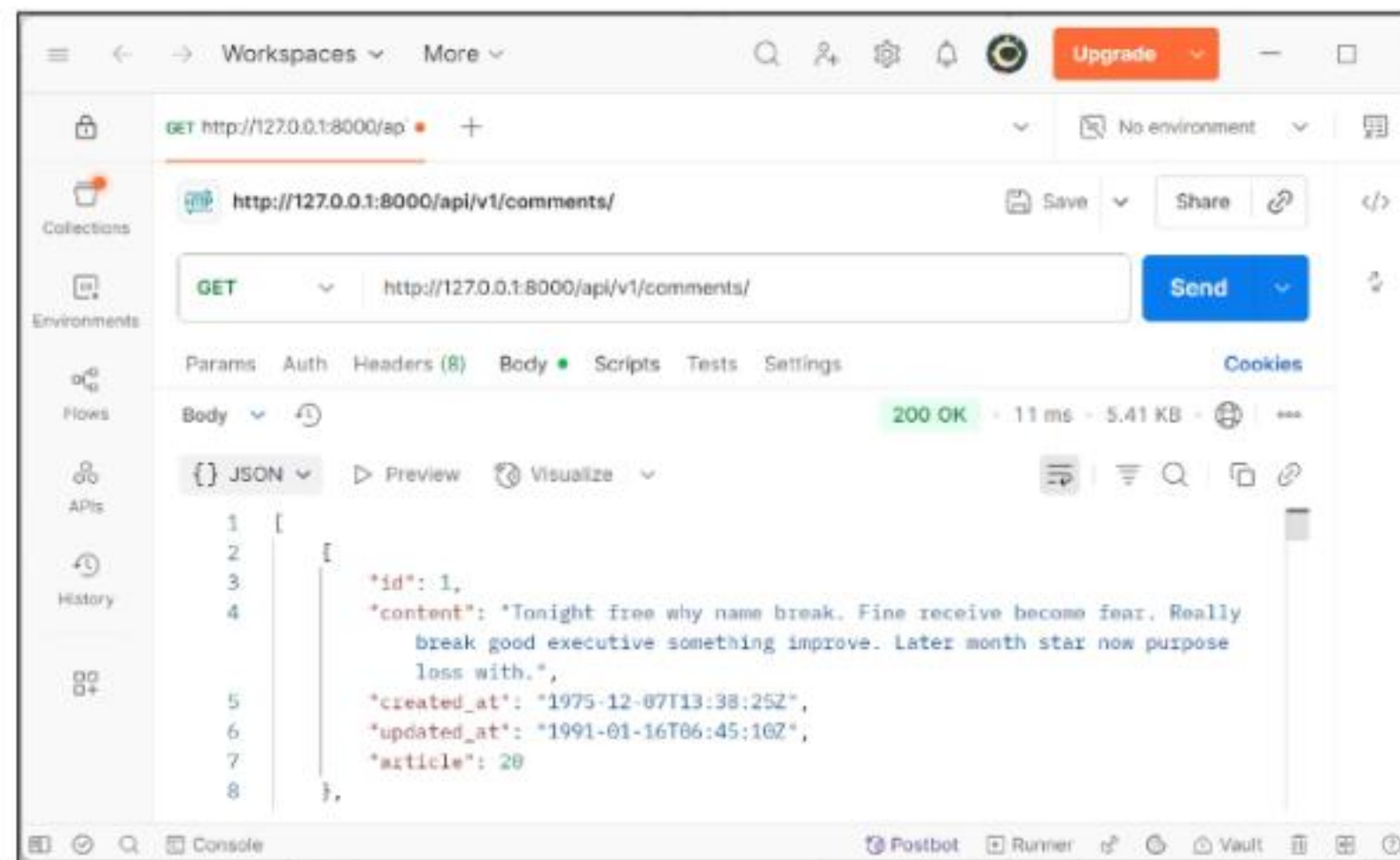
@api_view(['GET'])
def comment_list(request):
    comments = Comment.objects.all()
    serializer = CommentSerializer(comments, many=True)
    return Response(serializer.data)
```

GET - List (3/3)

- GET <http://127.0.0.1:8000/api/v1/comments/> 응답 확인



<그림1_Django_DRF 2_댓글 목록 조회 응답 확인>



<그림2_Django_DRF 2_댓글 목록 조회 응답 확인 POSTman>

GET - Detail (1/2)

- 단일 댓글 조회를 위한 url 및 view 함수 작성

```
# articles/urls.py

urlpatterns = [
    ...,
    path('comments/<int:comment_pk>/', views.comment_detail),
]
```

```
# articles/views.py

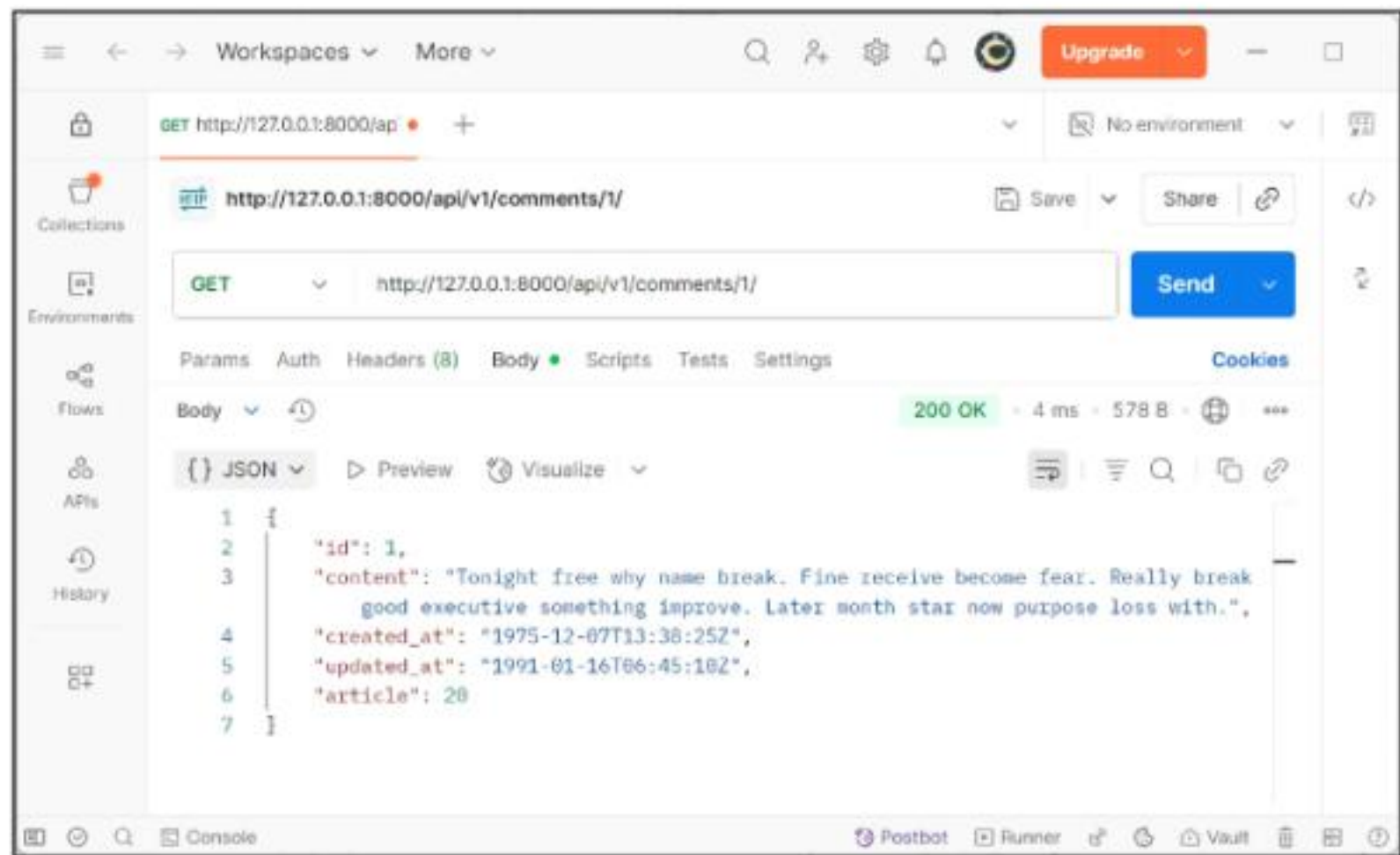
@api_view(['GET'])
def comment_detail(request, comment_pk):
    # 특정 댓글 데이터를 조회
    comment = Comment.objects.get(pk=comment_pk)
    serializer = CommentSerializer(comment)
    return Response(serializer.data)
```


GET - Detail (2/2)

- GET <http://127.0.0.1:8000/api/v1/comments/1/> 응답 확인



<그림3_Django_DRF 2_댓글 상세 조회 응답 확인>



<그림4_Django_DRF 2_댓글 상세 조회 응답 확인 POSTman>

POST method

POST (1/6)

- 단일 댓글 생성을 위한 url 및 view 함수 작성

```
# articles/urls.py

urlpatterns = [
    ...,
    path('articles/<int:article_pk>/comments/', views.comment_create),
]
```

```
# articles/views.py

@api_view(['POST'])
def comment_create(request, article_pk):
    # 어떤 게시글에 작성되는 댓글인지 단일 게시글을 조회
    article = Article.objects.get(pk=article_pk)
    serializer = CommentSerializer(data=request.data)
    if serializer.is_valid(raise_exception=True):
        serializer.save(article=article)
        return Response(serializer.data, status=status.HTTP_201_CREATED)
```


POST (2/6)

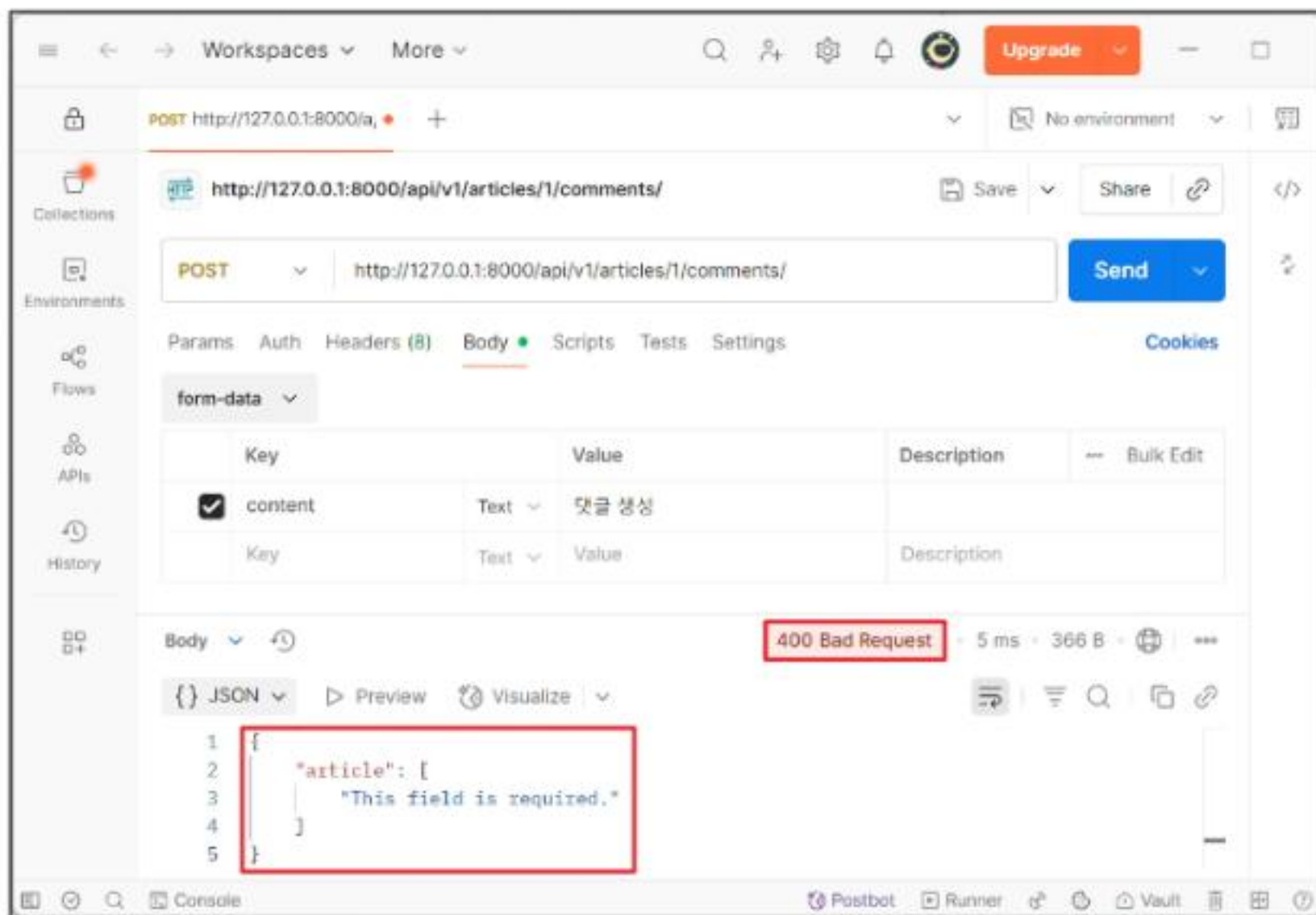
- serializer 인스턴스의 save() 메서드는 특정 Serializer 인스턴스를 저장하는 과정에서 추가 데이터를 받을 수 있음

```
# articles/views.py

@api_view(['POST'])
def comment_create(request, article_pk):
    # 어떤 게시글에 작성되는 댓글인지 단일 게시글을 조회
    article = Article.objects.get(pk=article_pk)
    serializer = CommentSerializer(data=request.data)
    if serializer.is_valid(raise_exception=True):
        # 추가 데이터를 save 메서드의 인자로 작성
        serializer.save(article=article)
    return Response(serializer.data, status=status.HTTP_201_CREATED)
```


POST (3/6)

- POST <http://127.0.0.1:8000/api/v1/articles/1/comments/> 응답 확인



<그림5_Django_DRF 2_댓글 생성 실패 POSTman>

```
{
  "article": [
    "This field is required."
  ]
}
```

400 Bad Request

※ 상태코드 400 응답 확인

POST (4/6)

- POST <http://127.0.0.1:8000/api/v1/articles/1/comments/> 응답 확인
 - 상태 코드 400 응답 확인
 - CommentSerializer에서 외래 키에 해당하는 article field 또한 사용자로부터 입력 받도록 설정되어 있기 때문에 서버 측에서는 누락되었다고 판단한 것
 - 유효성 검사 목록에서 제외 필요
 - article field를 **읽기 전용 필드**로 설정하기

**상태 코드**

요청 처리 결과를 알려주는 숫자
응답 신호

<개념2_Django_DRF 2_상태 코드>

POST (5/6)

- 데이터를 전송 받은 시점에

“**유효성 검사**에서 제외시키고, 데이터 조회 시에는 **출력**” 하는 필드

```
# articles/serializers.py

class CommentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Comment
        fields = 'all'
        read_only_fields = ('article',)
```

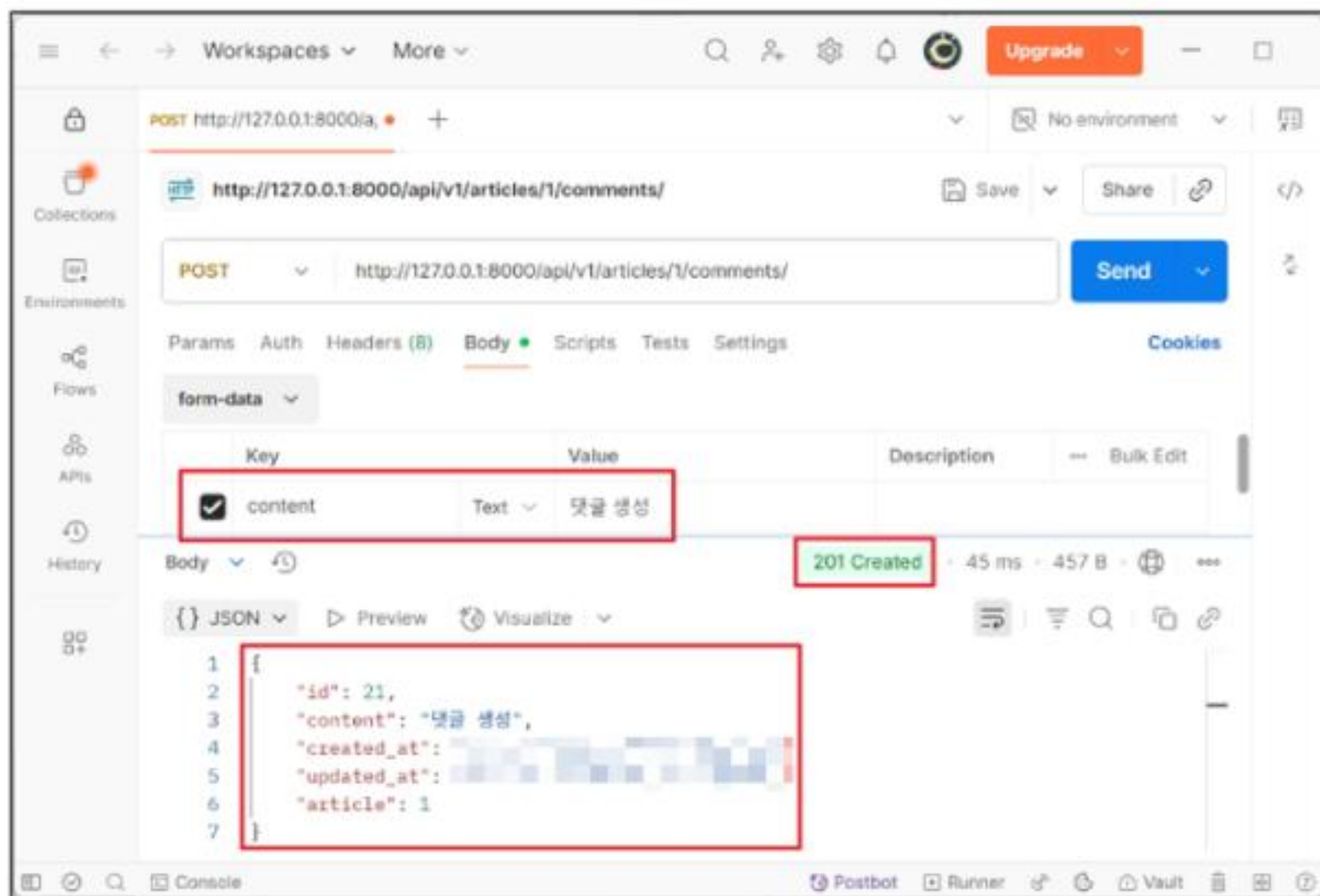


유효성 검사 (Validation)
입력된 데이터가 조건에 맞는지
확인하는 검사 과정

<개념3_Django_DRF 2_유효성 검사>

POST (6/6)

- POST <http://127.0.0.1:8000/api/v1/articles/1/comments/> 재요청



```
{
  "id": 21,
  "content": "댓글 생성",
  "created_at": "2023-07-10T10:10:10.101010Z",
  "updated_at": "2023-07-10T10:10:10.101010Z",
  "article": 1
}
```

201 Created

※ 상태코드 201 응답 확인

<그림6_Django_DRF 2_댓글 생성 성공 POSTman>

읽기 전용 필드

읽기 전용 필드 개념

읽기 전용 필드

`read_only_fields`

서버가 조회 요청에 대한
응답 시에만 값을 표시하는 필드

`read_only_fields`는 클라이언트가 입력해서는 안 되는 필드를 응답 전용 필드로 지정할 때 사용합니다.

- view에서 값을 직접 주입할 필드는 반드시 `read_only_fields`로 지정해 주세요.
- 그렇지 않으면 DRF는 해당 필드 값이 빠졌다고 판단해 400 에러를 발생시킵니다.

읽기 전용 필드 사용 목적

- 클라이언트 측에서 직접 수정하면 안 되는 경우
- 서버 로직에 의해 자동 생성·관리되는 값 활용
- 입력은 받지 않지만 정보를 제공해야 하는 경우
- 새로운 필드 값(추가 계산, 가공)을 만들어 제공해야 하는 경우

읽기 전용 필드 특징 및 주의사항

- 유효성 검사에서 제외됨
 - 읽기 전용 필드는 클라이언트가 보내는 요청 데이터에서 고려되지 않으므로, 유효성 검사 대상에서 제외됨
 - 즉, 클라이언트가 해당 필드에 값을 넣어도 무시되며 검증 오류를 일으키지 않음
- 생성·수정 요청 모두에서 적용 가능
 - 읽기 전용 필드라 해서 생성(POST) 단계에서만 무의미한 것은 아님
 - 수정(PUT) 요청에서도 해당 필드는 여전히 클라이언트 입력을 받지 않고, 응답 시에만 노출됨

DELETE & PUT method

DELETE & PUT (1/3)

- 단일 댓글 삭제 및 수정을 위한 view 함수 작성

```
# articles/views.py
@api_view(['GET', 'PUT', 'DELETE'])
def comment_detail(request, comment_pk):
    comment = Comment.objects.get(pk=comment_pk)
    if request.method == 'GET':
        serializer = CommentSerializer(comment)
        return Response(serializer.data)

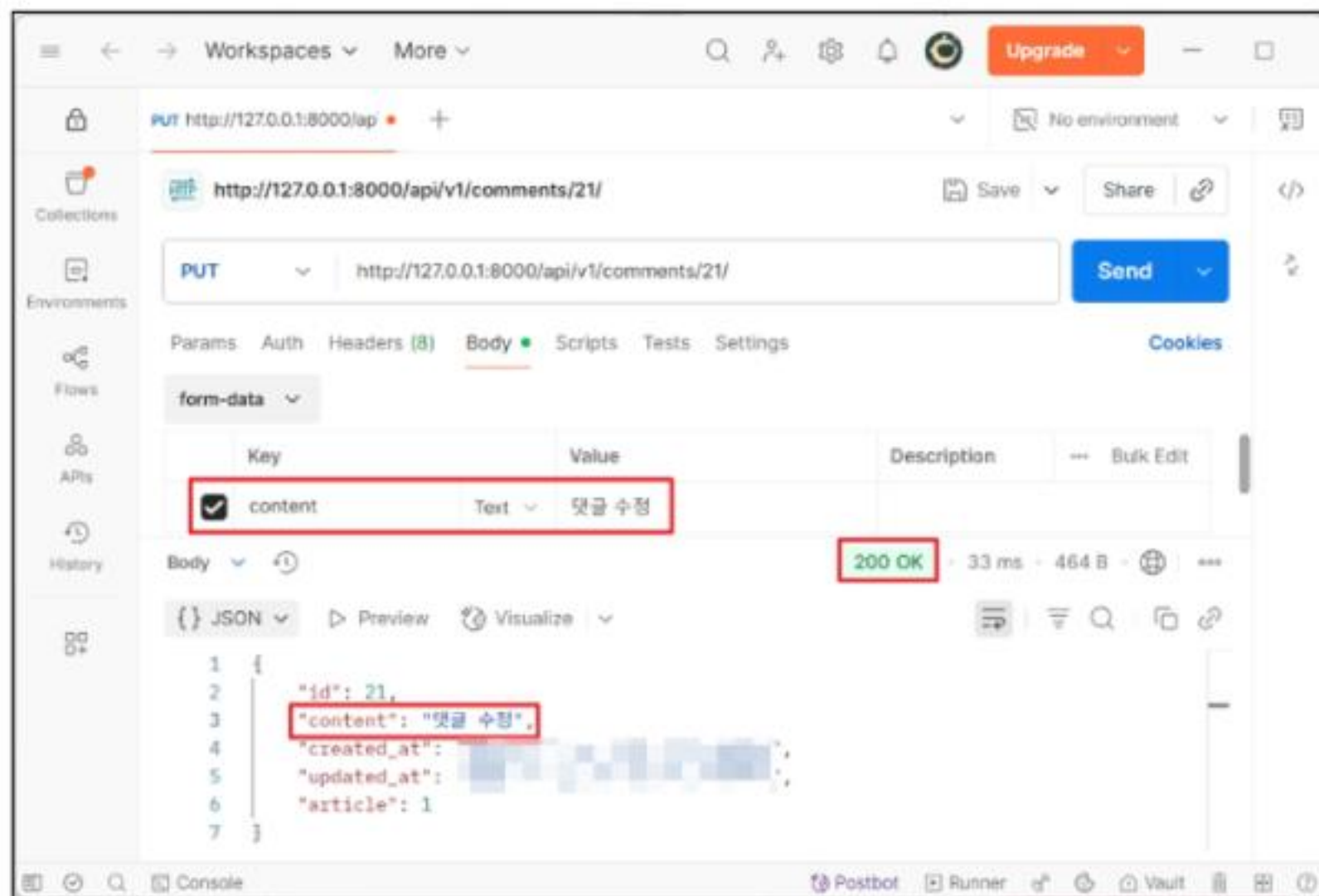
    elif request.method == 'PUT':
        serializer = CommentSerializer(comment, data=request.data)
        if serializer.is_valid(raise_exception=True):
            serializer.save()
            return Response(serializer.data)

    elif request.method == 'DELETE':
        comment.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

DELETE & PUT (2/3)

※ comment pk는 각자 상황마다 다를 수 있음

- PUT http://127.0.0.1:8000/api/v1/comments/{{comment_pk}}/ 응답 확인



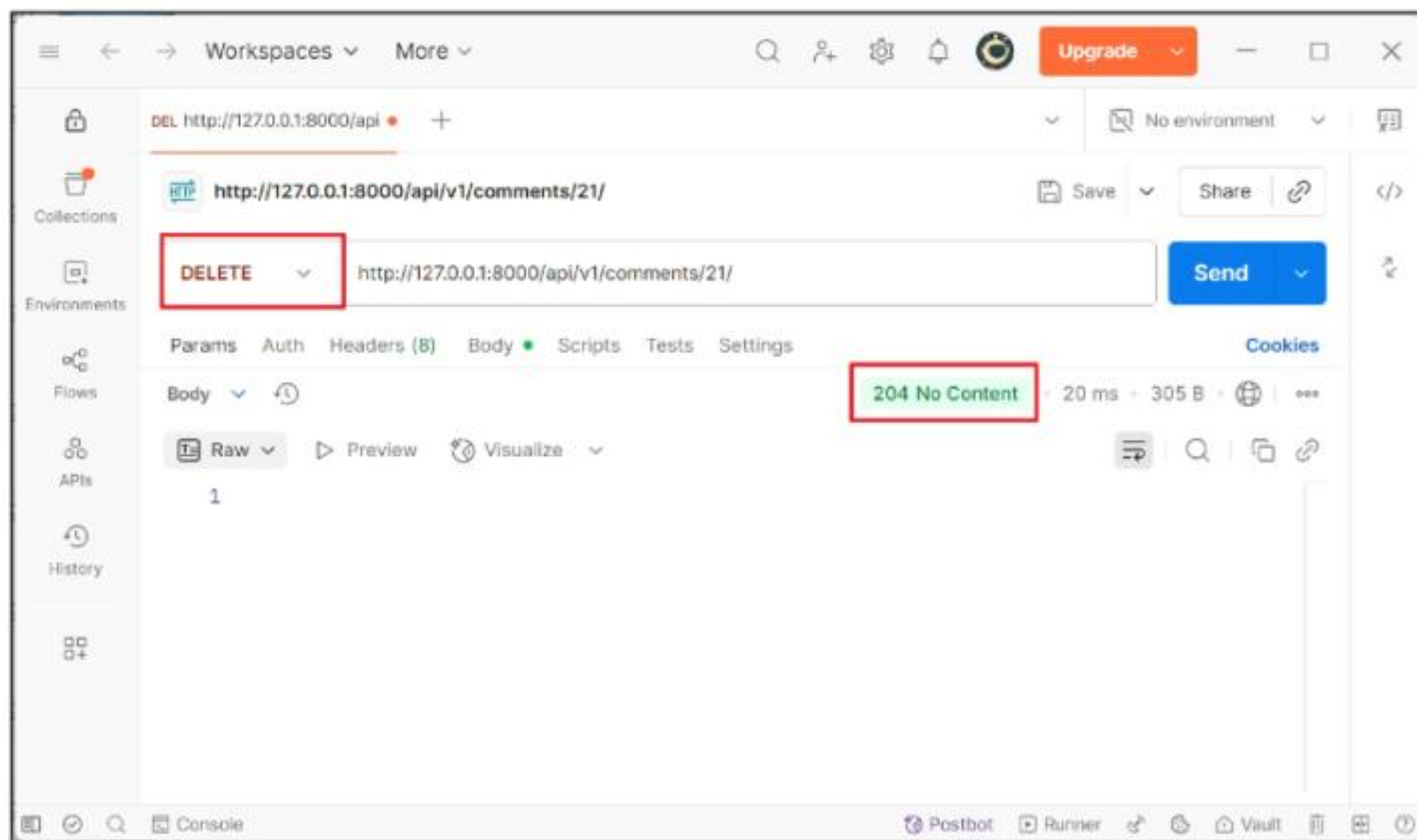
form-data			
	Key	Value	
☒	content	Text	댓글 수정

```
{  
  "id": 21,  
  "content": "댓글 수정",  
  "created_at": ...,  
  "updated_at": ...,  
  "article": 1  
}
```


DELETE & PUT (3/3)

※ comment pk는 각자 상황마다 다를 수 있음

- DELETE `http://127.0.0.1:8000/api/v1/comments/{{ comment_pk }}/` 응답 확인



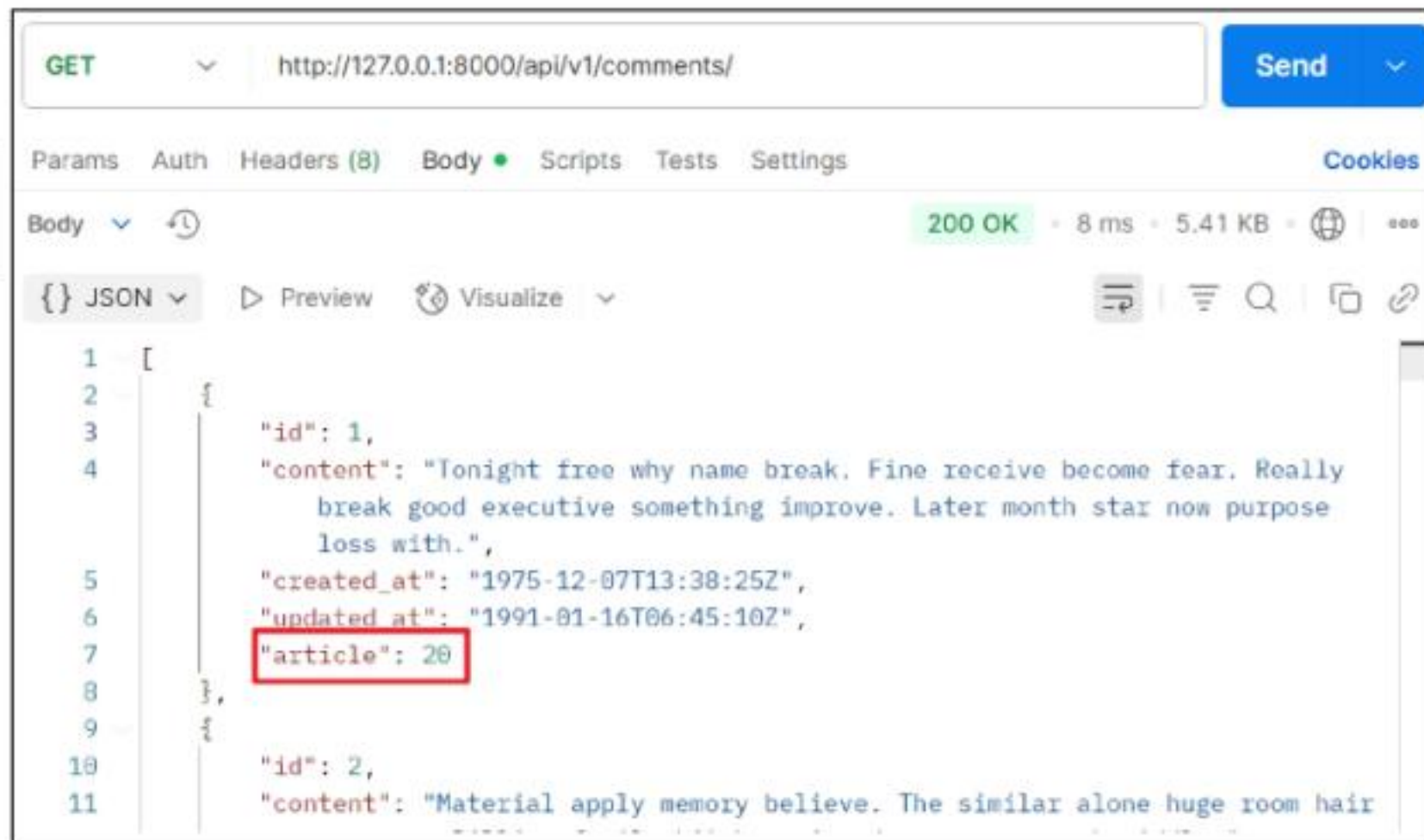
204 No Content

※ 상태코드 204 응답 확인

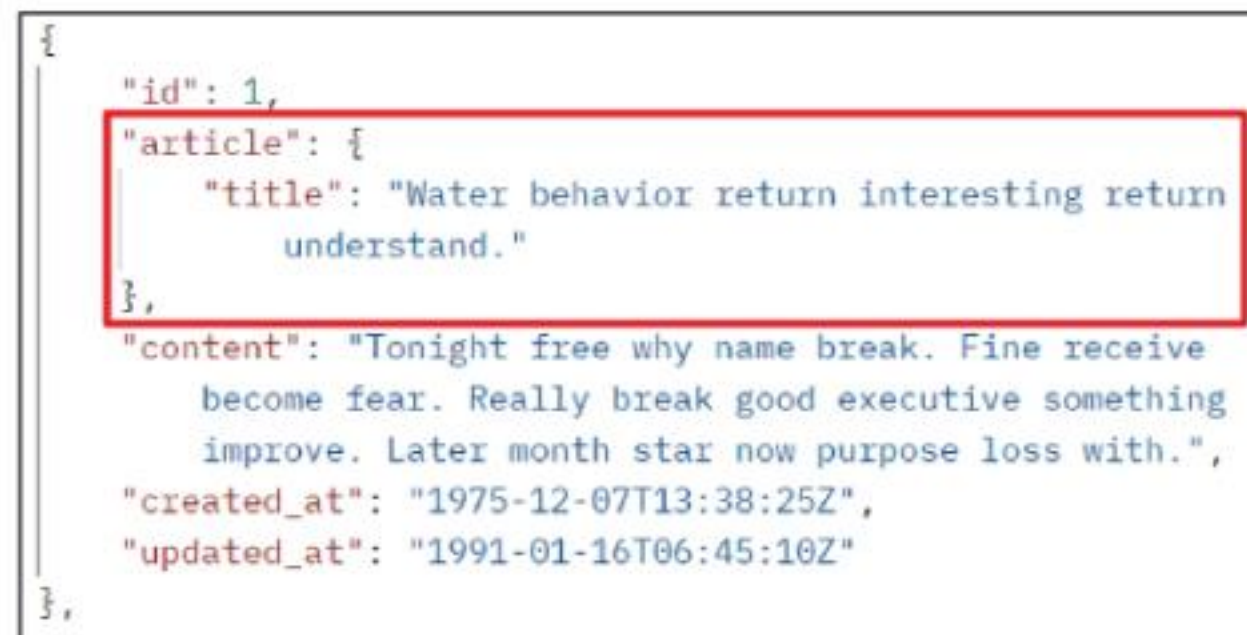
응답 데이터 재구성

댓글 조회 시 게시글 출력 내역 변경 (1/5)

- 댓글 목록 조회 시 게시글 번호만 제공해주는 것이 아닌 **'게시글의 제목'**까지 제공하기



<그림9_Django_DRF 2_기존 댓글 목록 조회 POSTman>



<그림10_Django_DRF 2_게시글 제목을 포함한 댓글 조회 POSTman>

댓글 조회 시 게시글 출력 내역 변경 (2/5)

- Comment 모델은 Article을 참조하고 있음
 - Comment가 article과 N:1 관계를 맺고 있고, Django는 기본적으로 이 관계를 통해 숫자(id) 값만 응답에 포함

```
# articles/models.py
class Comment(models.Model):
    article = models.ForeignKey(Article, on_delete=models.CASCADE)
    ...
```

- CommentSerializer는 Comment의 정보를 가지고 있음
 - Comment 모델이 Article을 참조하고 있어, 그 정보를 id로 field에 제공할 수 있는 것

```
# articles/serializers.py
class CommentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Comment
    ...
```


댓글 조회 시 게시글 출력 내역 변경 (3/5)

- Serializer는 DRF에서 응답 구조를 결정하는 주체
 - Serializer를 통해 어떤 필드를 포함할지 직접 지정하여 사용하고 있음
- ```
fields = ('id', 'title', 'content',)
```
- 또한, 특정 필드를 어떤 형식으로 보여줄지도 지정하여 사용 할 수 있음
  - 즉, article 필드가 “id가 아닌 게시글의 제목”을 보여 주도록 지정 할 수도 있음
- 하지만, CommentSerializer는 Comment의 정보만 가지고 있을 뿐, Article에 대한 정보는 없음
    - article의 정보를 포함하고 있고, 그 중 title 필드의 정보만 반환하는 Serializer를 별도로 정의해야 함

```
articles/serializers.py
class ArticleTitleSerializer(serializers.ModelSerializer):
 class Meta:
 model = Article
 fields = ('title',)
```



## 댓글 조회 시 게시글 출력 내역 변경 (4/5)

- ArticleTitleSerializer는 어디에 정의해야 할까?
  - CommentSerializer에서만 사용할 용도라면, 굳이 독립적으로 선언할 필요 없음
  - 코드의 응집도를 높이고, 명확한 범위를 지정하기 위해서 CommentSerializer 내부에 정의
- Comment 모델의 article 필드를 ArticleTitleSerializer로 재정의
  - 이제, article 필드는 게시글의 title 필드만 포함한 구조로 응답

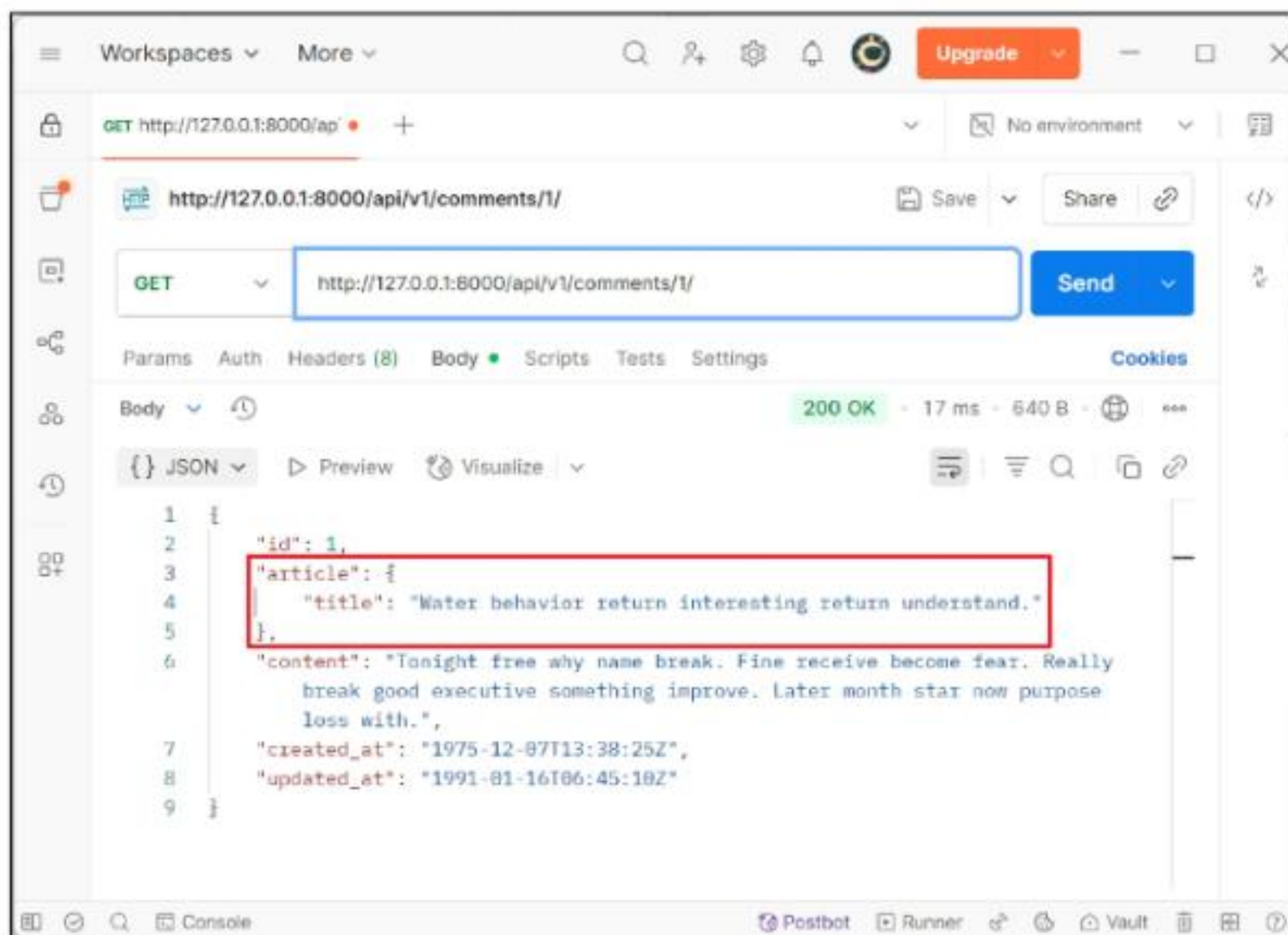
```
articles/serializers.py
class CommentSerializer(serializers.ModelSerializer):
 class ArticleTitleSerializer(serializers.ModelSerializer):
 class Meta:
 model = Article
 fields = ('title',)

 article = ArticleTitleSerializer(read_only=True)

 class Meta:
 model = Comment
 fields = '__all__'
 # read_only_fields = ('article',)
```

## 댓글 조회 시 게시글 출력 내역 변경 (5/5)

- GET <http://127.0.0.1:8000/api/v1/comments/1/> 응답 확인



<그림11\_Django\_DRF 2\_댓글 조회 시 게시글 출력 내역 변경>

# 읽기 전용 필드 주의사항



## 읽기 전용 필드 지정 주의사항

- 특정 필드를 재정의 혹은 추가한 경우 read\_only\_fields는 동작하지 않음
- 이런 경우 새로운 필드에 read\_only 키워드 인자로 작성해야 함

```
articles/serializers.py
class CommentSerializer(serializers.ModelSerializer):
 class ArticleTitleSerializer(serializers.ModelSerializer):
 class Meta:
 model = Article
 fields = ('title',)

 article = ArticleTitleSerializer(read_only=True)

 class Meta:
 model = Comment
 fields = '__all__'
 # read_only_fields = ('article',)
```





## read\_only\_fields 속성과 read\_only 인자

- read\_only\_fields

- 기존 외래 키 필드 값을 그대로 응답 데이터에 제공하기 위해 지정하는 경우

- read\_only

- 기존 외래 키 필드 값의 결과를 다른 값으로 덮어쓰는 경우
  - 새로운 응답 데이터 값을 제공하는 경우

# 역참조 데이터 구성

## Article → Comment 간 역참조 관계를 활용한 JSON 데이터 재구성

- 아래 2가지 사항에 대한 데이터 재구성하기

1. 단일 게시글 조회 시 해당 게시 글에 작성된 댓글 목록도 함께 붙여서 응답
2. 단일 게시글 조회 시 해당 게시 글에 작성된 댓글 개수도 함께 붙여서 응답

# 단일 게시물 + 댓글 목록



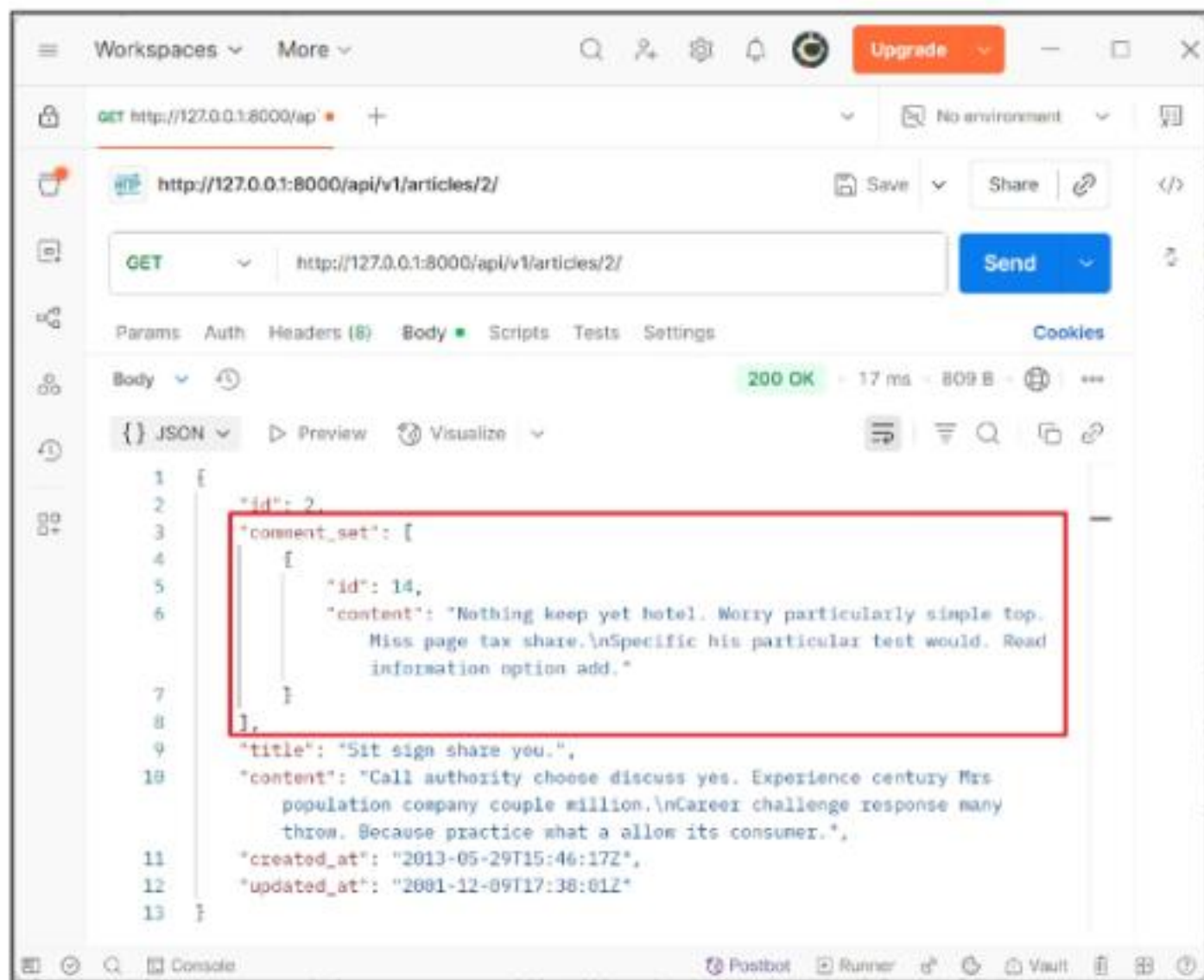
## 1. 단일 게시글 + 댓글 목록 (1/2)

- Nested relationships (역참조 매니저 활용)
  - 모델 관계 상으로 참조하는 대상(N)은 참조되는 대상(1)의 표현에도 포함되거나 중첩될 수 있음
  - Comment가 Article에 대한 정보를 article field를 사용하여 표현하였 듯, Article은 자신을 참조하고 있는 comment들에 대한 정보를 역참조 매니저를 통해 표현 할 수 있음

```
class ArticleSerializer(serializers.ModelSerializer):
 class CommentDetailSerializer(serializers.ModelSerializer):
 class Meta:
 model = Comment
 fields = ('id', 'content',)
 comment_set = CommentDetailSerializer(many=True, read_only=True)
 class Meta:
 model = Article
 fields = '__all__'
```

## 1. 단일 게시물 + 댓글 목록 (2/2)

- GET <http://127.0.0.1:8000/api/v1/articles/2/> 응답 확인



```

"comment_set": [
 {
 "id": 14,
 "content": "Nothing keep yet hotel. Worry particularly simple top.
 Miss page tax share.\nSpecific his particular test would. Read
 information option add."
 }
],

```

# 단일 게시물 + 댓글 개수

## 단일 게시물 조회 시, 댓글 개수도 함께 제공하고 싶다면?

- 기본적으로 게시물(Article)을 조회하면 참조 중인 댓글(Comment)의 개수는 알 수 없음
  - Comment모델과의 관계는 Article.comment\_set으로 연결되지만,  
댓글의 개수를 저장하는 별도 필드는 Article 모델에 정의 한 적 없기 때문
- 따라서, 댓글 수를 응답하려면, 직접 계산해서 응답에 포함시켜야 함



## View 로직 개선: annotate 사용

- View에서 Article 객체를 조회할 때 annotate를 활용해 num\_of\_comments 필드를 추가
  - ※ annotate는 Django ORM 함수로, SQL의 집계 함수를 활용하여 쿼리 단계에서 데이터 가공을 수행
- View에서 Article 객체를 조회할 때 annotate를 활용해 num\_of\_comments 필드를 추가
- 다음과 같이 댓글 수를 세어 num\_of\_comments라는 필드를 추가
  - 이제 serializer.data가 반환하는, article 객체에는 num\_of\_comments라는 "주석(annotate) 필드"가 포함되어 있음

```
articles/views.py

@api_view(['GET', 'DELETE', 'PUT'])
def article_detail(request, article_pk):
 article = Article.objects.annotate(num_of_comments=Count('comment')).get(pk=article_pk)
```

- ※ Count('comment') 에서 'comment'는 Article을 참조하고 있는 모델 Comment의 소문자 표기

## annotate를 사용하여 추가한 "주석 필드"를 serializer에 추가하려면?

- 단순히 `fields = '__all__'` 만으로는 annotate된 필드가 포함되지 않음
    - `annotate()`는 실제 모델 필드를 생성하지 않기 때문
    - `annotate()`는 쿼리 시점에만 존재하는 **임시 필드를 추가하는 기능**
    - 즉, `Article` 모델 클래스에는 `num_of_comments`라는 필드가 실제로 존재하지 않음
  - `__all__`은 **모델의 필드 기준으로 작동**하기 때문에, `annotate`로 만들어진 필드는 여기에 포함되지 않음
  - 이런 동적으로 계산된 필드를 응답에 포함하려면, `SerializerMethodField`를 사용해야 함
- ※ `SerializerMethodField` 필드에 대한 자세한 설명은 코드 작성 후 진행



## Serializer 개선: SerializerMethodField 사용 (1/2)

- SerializerMethodField는 **읽기 전용 필드를 커스터마이징** 하는데 사용
- 이 필드를 선언한 뒤 get\_<필드명> 메서드를 정의하면, 해당 메서드의 반환 값이 직렬화 결과에 포함됨

```
articles/serializers.py
class ArticleSerializer(serializers.ModelSerializer):
 ...
 num_of_comments = serializers.SerializerMethodField()

 class Meta:
 ...

 def get_num_of_comments(self, obj):
 # 여기서 obj는 Serializer가 처리하는 Article 인스턴스
 # view에서 annotate 한 필드를 그대로 사용 가능
 return obj.num_of_comments
```

## Serializer 개선: SerializerMethodField 사용 (2/2)

- 이제 `serializer.data` 호출시, `get_num_of_comments` 메서드가 실행되어 `num_of_comments` 값이 자동으로 응답 할 데이터 필드에 포함되어 제공 됨
- 추가적으로 view 에서 `data`를 딕셔너리로 변환하거나 수정할 필요 없이, `serializer.data`를 바로 반환 해도 최종 JSON 응답에 `num_of_comments` 값이 반영됨

```
articles/serializers.py
class ArticleSerializer(serializers.ModelSerializer):
 ...
 num_of_comments = serializers.SerializerMethodField()

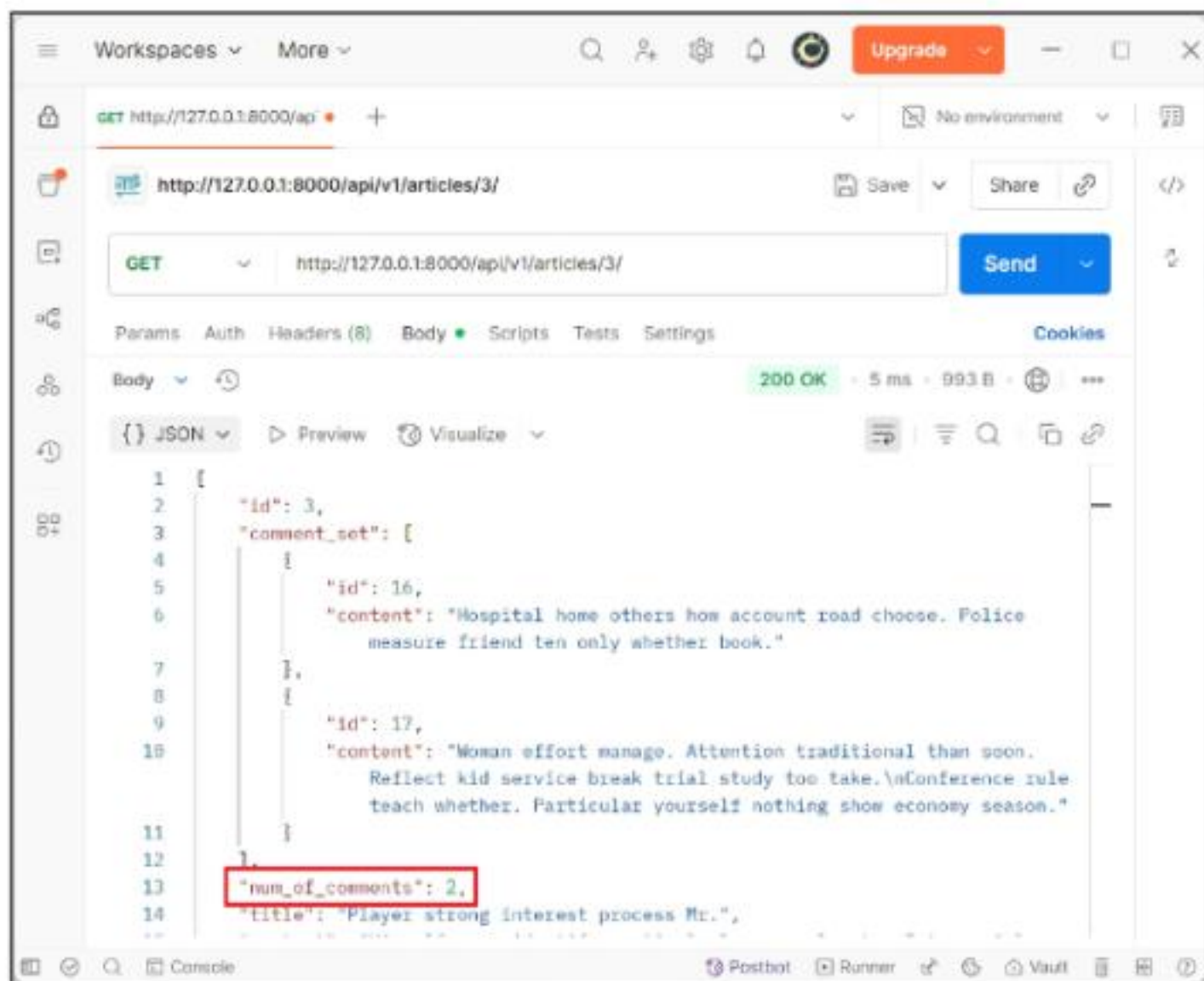
 class Meta:
 ...

 def get_num_of_comments(self, obj):
 # 여기서 obj는 Serializer가 처리하는 Article 인스턴스
 # view에서 annotate 한 필드를 그대로 사용 가능
 return obj.num_of_comments
```



## 댓글 개수 데이터 응답 확인

- GET <http://127.0.0.1:8000/api/v1/articles/3/> 응답 확인



"num\_of\_comments": 2,

# SerializerMethodField

## SerializerMethodField란?

# SerializerMethodField

Serializer에서 추가적인 데이터  
가공을 하고 싶을 때 사용

예를 들어, 특정 필드 값을 조합해 새로운 문자열 필드를 만들거나,  
부가적인 계산 (비율, 합계, 평균)을 하는 경우에 활용할 수 있습니다!

## SerializerMethodField 동작 원리 (1/2)

- SerializerMethodField를 Serializer 클래스 내에서 필드로 선언하면, DRF는 get\_<필드명>이라는 이름을 가진 메서드를 자동으로 찾음
- 예를 들어, full\_name = serializers.SerializerMethodField()라고 선언하면, DRF는 get\_full\_name(self, obj) 메서드를 찾아 해당 값을 직렬화 결과에 넣어 줌

```
class UserSerializer(serializers.ModelSerializer):
 full_name = serializers.SerializerMethodField()

 class Meta:
 model = User
 fields = ('id', 'username', 'full_name', 'email',)

 def get_full_name(self, obj):
 return f'{obj.first_name} {obj.last_name}'
```



## SerializerMethodField 동작 원리 (2/2)

- obj는 현재 직렬화 중인 모델 인스턴스이며, 이 메서드에서 obj의 속성이나 annotate된 필드를 활용해서 값을 만들 수 있음

```
class UserSerializer(serializers.ModelSerializer):
 full_name = serializers.SerializerMethodField()

 class Meta:
 model = User
 fields = ('id', 'username', 'full_name', 'email',)

 def get_full_name(self, obj):
 return f'{obj.first_name} {obj.last_name}'
```

## SerializerMethodField 주의사항

- 읽기 전용
- 생성(POST), 수정(PUT) 요청 시에는 사용되지 않음
- get\_ 메서드는 반드시 (self, obj) 형태로 정의해야 하며, obj는 현재 직렬화 중인 모델 인스턴스를 의미

```
class UserSerializer(serializers.ModelSerializer):
 full_name = serializers.SerializerMethodField()

 class Meta:
 model = User
 fields = ('id', 'username', 'full_name', 'email',)

 def get_full_name(self, obj):
 return f'{obj.first_name} {obj.last_name}'
```

## SerializerMethodField 사용 목적

- 유연성
  - 다양한 계산 로직을 손쉽게 추가 가능
- 가독성
  - 데이터 변환 과정을 Serializer 내부 메서드로 명확히 분리
- 유지보수성
  - view나 model에 비해 Serializer 측 로직 변경이 용이
- 일관성
  - view에서 별도로 data 수정 없이도 직렬화 결과를 제어

# 역참조 데이터 구성 참고



## 역참조 매니저 활용 참고

※ 만약, 역참조 매니저명을 변경하였다면, Serializer에서도 변경하여야 함

```
articles/models.py
class Comment(models.Model):
 article = models.ForeignKey(Article, on_delete=models.CASCADE, related_name='comments')
 ...
```

```
class ArticleSerializer(serializers.ModelSerializer):
 class CommentDetailSerializer(serializers.ModelSerializer):
 ...

 comments = CommentDetailSerializer(many=True, read_only=True)
 ...
```

```
"id": 2,
"comments": [
 {
 "id": 14,
 "content": "Nothing keep yet hotel. Worry particularly simple top.
Miss page tax share.\nSpecific his particular test would. Read
information option add."
 }
],
```

## annotate 활용 참고

※ 만약, 역참조 매니저명을 변경하였다면, annotate에서도 변경하여야 함

```
articles/models.py
class Comment(models.Model):
 article = models.ForeignKey(Article, on_delete=models.CASCADE, related_name='comments')
 ...
```

```
articles/views.py
@api_view(['GET', 'DELETE', 'PUT'])
def article_detail(request, article_pk):
 article = get_object_or_404(Article.objects.annotate(num_of_comments=Count('comments')), pk=article_pk)
```

# API 문서화

# OpenAPI Specification



# OpenAPI Specification

OAS

RESTful API를 설명하고 시각화하는 표준화된 방법

- API에 대한 세부사항을 기술할 수 있는 공식 표준

## RESTful API를 설명하고 시각화하는 표준화된 방법



<그림14\_Django\_DRF 2\_Swagger 로고>



<그림15\_Django\_DRF 2\_Redoc 로고>

OAS 기반 API에 대한 문서를 생성하는 데  
도움을 주는 오픈소스 프레임워크

# 문서화 활용

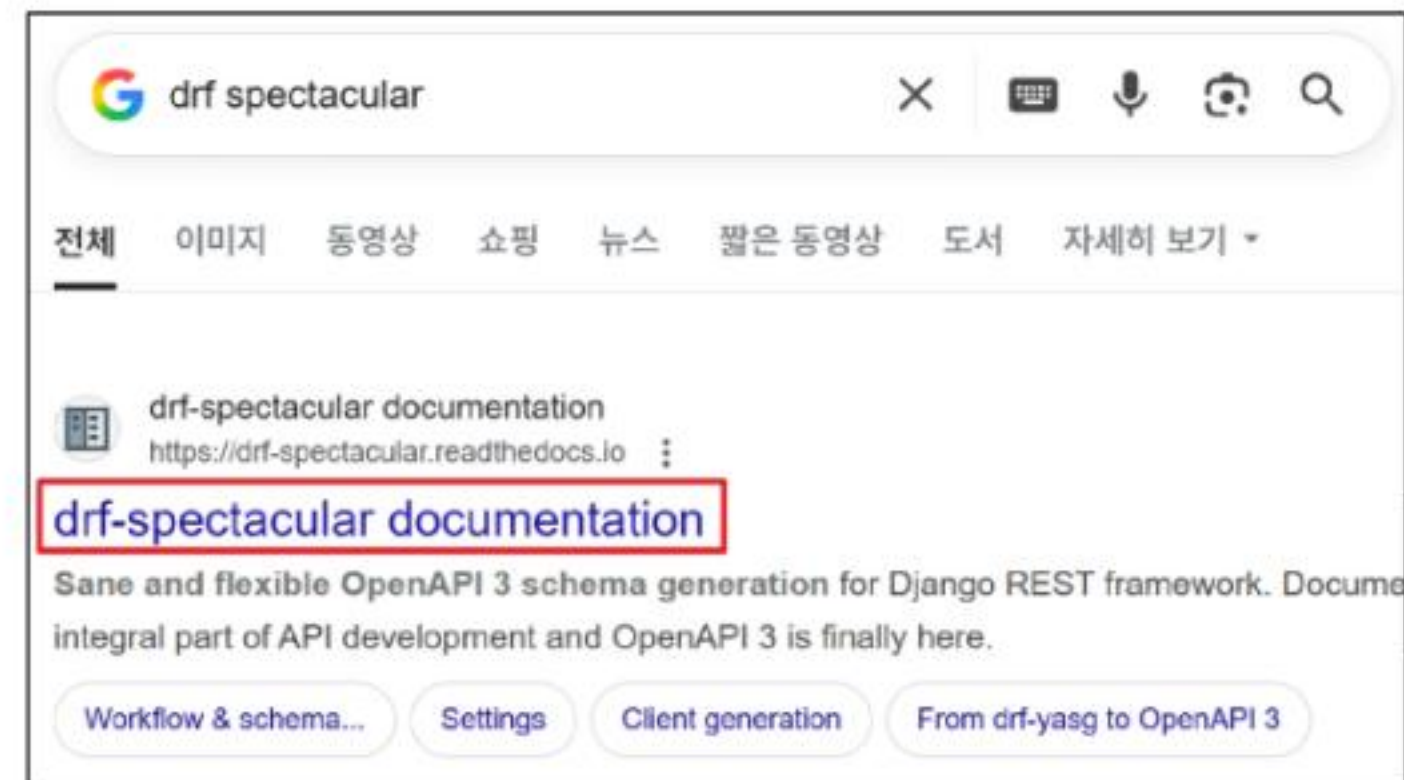
## drf-spectacular 라이브러리 (1/4)

- DRF 위한 OpenAPI 3.0 구조 생성을 도와주는 라이브러리
- 설치 및 등록

```
$ pip install drf-spectacular
```

```
settings.py
```

```
INSTALLED_APPS = [
 ...,
 'drf_spectacular',
 ...,
]
```



〈그림16\_Django\_DRF 2\_DRF Spectacular 공식 문서〉

※ DRF Spectacular 공식 문서 참고 필수!



## drf-spectacular 라이브러리 (2/4)

- 관련 설정 코드 입력 (OpenAPI 구조 자동 생성 코드)

```
settings.py

REST_FRAMEWORK = {
 # YOUR SETTINGS
 'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
}
```

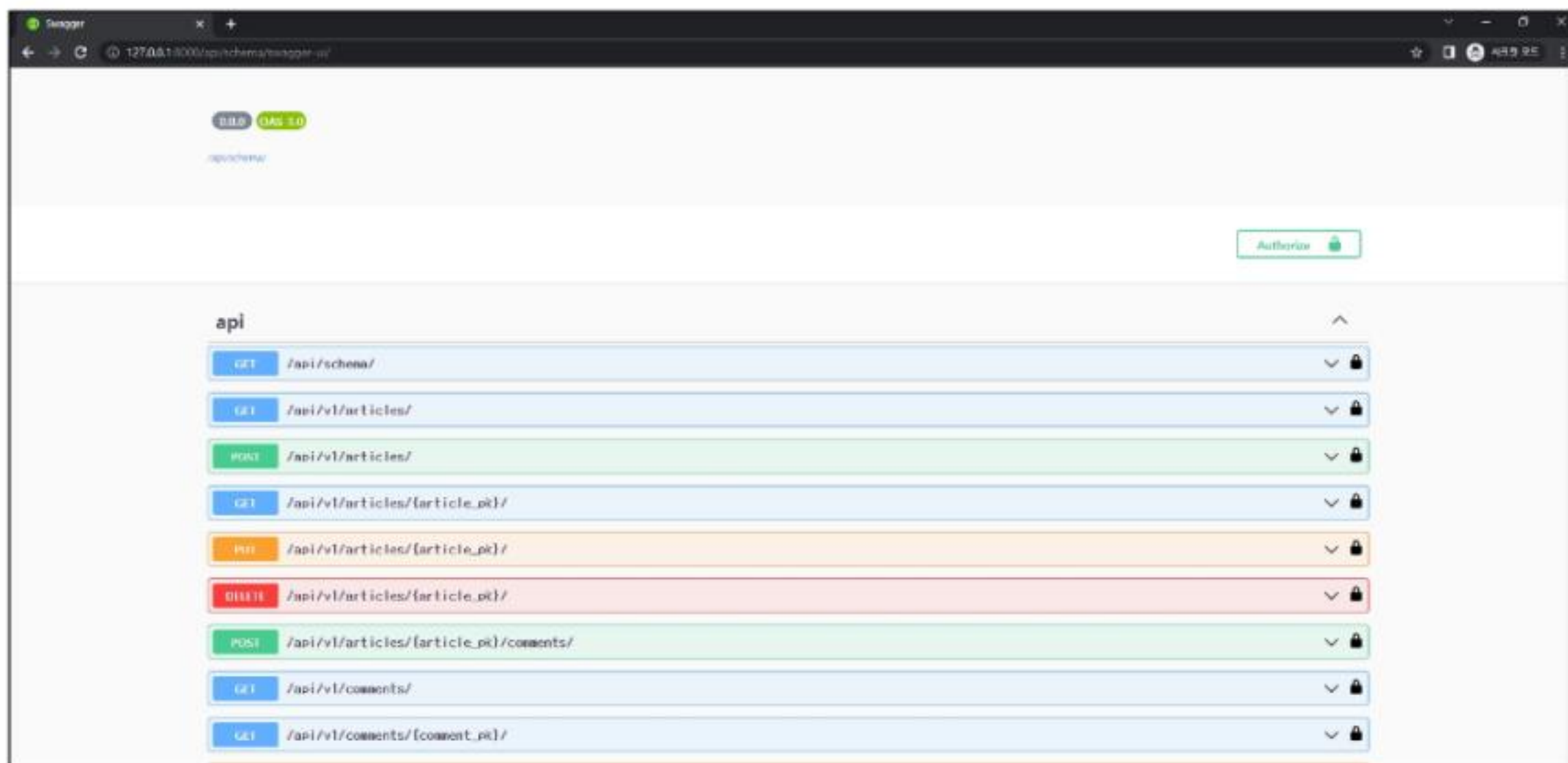
- swagger, redoc 페이지 제공을 위한 url 작성

```
drf/urls.py
from drf_spectacular.views import SpectacularAPIView, SpectacularRedocView, SpectacularSwaggerView

urlpatterns = [
 ...,
 # Optional UI:
 path('api/schema/swagger-ui/', SpectacularSwaggerView.as_view(url_name='schema'), name='swagger-ui'),
 path('api/schema/redoc/', SpectacularRedocView.as_view(url_name='schema'), name='redoc'),
]
```

## drf-spectacular 라이브러리 (3/4)

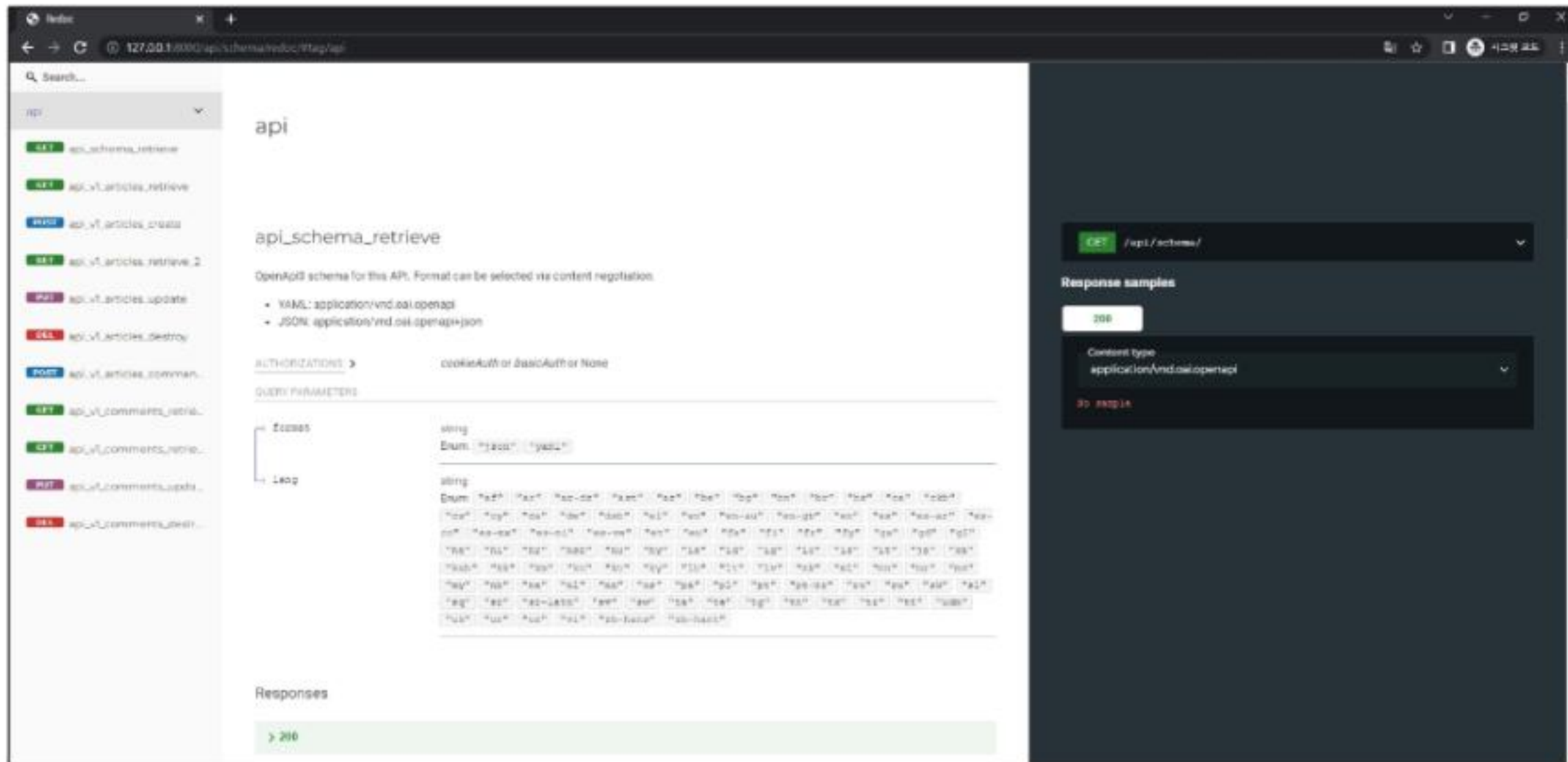
- <http://127.0.0.1:8000/api/schema/swagger-ui/> 페이지 확인



<그림17\_Django\_DRF 2\_swagger ui 화면>

## drf-spectacular 라이브러리 (4/4)

- <http://127.0.0.1:8000/api/schema/redoc/> 페이지 확인



〈그림18\_Django\_DRF 2\_redoc 화면〉

## "설계 우선" 접근법

- OAS의 핵심 이점
  - API를 먼저 설계하고 명세를 작성한 후, 이를 기반으로 코드를 구현하는 방식
  - API의 일관성을 유지하고, API 사용자는 더 쉽게 API를 이해하고 사용할 수 있음
  - 또한, OAS를 사용하면 API가 어떻게 작동하는지를 시각적으로 보여주는 문서를 생성할 수 있으며, 이는 API를 이해하고 테스트하는 데 매우 유용
- 이런 목적으로 사용되는 도구가 Swagger-UI 또는 ReDoc



# 참고

# 올바르게 404 응답하기

## HTTP Response Shortcuts

# Django shortcuts functions

`render()`

`redirect()`

`get_object_or_404()`

`get_list_or_404()`

## HTTP Response Shortcuts

# get\_object\_or\_404()

모델 manager objects에서 `get()`을 호출하지만,  
해당 객체가 없을 땐 기존 `DoesNotExist` 예외 대신  
`Http404`를 `raise` 함



## get\_object\_or\_404 적용 예시

```
articles/views.py
article = Article.objects.get(article_pk=article_pk)
comment = Comment.objects.get(comment_pk=comment_pk)
```



```
articles/views.py

from django.shortcuts import get_object_or_404

article = get_object_or_404(Article, pk=article_pk)
article = get_object_or_404(
 Article.objects.annotate(num_of_comments=Count('comment')),
 pk=article_pk,
)
comment = get_object_or_404(Comment, pk=comment_pk)
```

### • 기존 작성 방식

- 조회 대상이 없는 경우, 500 Server Error 반환

### • get\_object\_or\_404 적용

- 조회 대상이 없는 경우, 404 Not Found 반환

## HTTP Response Shortcuts

# get\_list\_or\_404()

모델 manager objects에서 filter()의 결과를 반환하고,  
해당 객체 목록이 없을 땐 **Http404를 raise** 함

## get\_list\_or\_404 적용 예시

```
articles/views.py
article = Article.objects.all()
comment = Comment.objects.all()
```



```
articles/views.py

from django.shortcuts import get_list_or_404

article = get_list_or_404()
comment = get_list_or_404()
```

### • 기존 작성 방식

- 조회 대상 목록이 없어도, 200 OK 반환

### • get\_list\_or\_404 적용

- 조회 대상이 없는 경우, 404 Not Found 반환



## 왜 사용해야 할까?

- `get_object_or_404()`를 사용하지 않은 경우
  - 클라이언트에게 “서버에 오류가 발생하여 요청을 수행할 수 없다”라는 원인이 정확하지 않은 에러를 제공한 경우,  
클라이언트의 요청이 올바르지 않았음에도 서버의 문제로 오해할 수 있음
- `get_list_or_404()`를 사용하지 않은 경우
  - 조건에 맞는 데이터가 하나도 없는데도 단순히 “빈 리스트를 반환”하면, 클라이언트는 요청이 올바르지 않았는지,  
아니면 진짜 데이터가 없는 건지 명확하게 판단하기 어려움
- 적절한 예외 처리를 통해 클라이언트에게 보다 정확한 에러 현황을 전달하는 것은 매우 중요한 개발 요소임
  - 조건에 맞는 객체가 하나도 없을 경우, 명확하게 404 응답을 반환하여 함
  - 이를 통해, 클라이언트에게 “해당 조건에 맞는 리소스가 존재하지 않는다”는 정확한 상황을 전달할 수 있음



# View와 Serializer의 역할

## View와 Serializer

- view나 queryset 로직에서는 비즈니스 로직(데이터 가공, annotate, 필터링)을 처리
- serializer는 그 결과물을 직렬화하는 역할에 집중하는 것이 일반적인 권장사항
- 복잡한 query나 로직은 **View 함수에서 진행**
  - 여러 모델을 조인하거나 복잡한 집계가 필요한 경우 View 함수에서 처리
  - 필요한 경우 View 함수에서 `select_related()`나 `prefetch_related()`를 사용하여 query를 최적화

# DRF 학습 이유

## 왜 DRF를 배웠을까?

- 백엔드와 프론트엔드의 분리 경험

- 기존 Django 템플릿 기반의 서버 렌더링 방식을 벗어나,  
백엔드(데이터·로직)와 프론트엔드(UI)를 명확히 분리하는 패턴을 간접적으로 체험

- 표준화된 API 구축 역량 확보

- DRF를 통해 RESTful API를 손쉽게 만들고 관리하는 방법을 학습했는데,  
이는 다양한 클라이언트(웹, 모바일 앱, 외부 서비스)와 연동하는 데 필수적인 능력

- 프론트엔드 기술과의 연결 고리

- 앞으로 학습할 Javascript 및 Vue는 주로 API를 통해 데이터를 받아와 화면을 구성함
- DRF로 구축한 일관된 API는 Vue 등 프론트엔드 프레임워크와 매끄럽게 호환됨



## DRF with N:1 Relation

- 2946. 도서 관리 API 만들기 - 도서 리뷰 모델 정의
- 2948~2952. 충전소 관리 API 서비스 만들기
- 3067. DRF 서버에 POSTman으로 요청 보내기
- 3068. DRF에서 CRUD 구현하기

## 역참조 데이터 구성

- 2947. 도서 관리 API 만들기 - 제공 정보 개선
- 3069. DRF에서 1:N 모델 구축하기

## Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 중 댓글(Comment)이 게시글(Article)과 N:1 관계를 나타내는 모델 필드 정의는?

- a) `article = models.OneToOneField(Article, ...)`
- b) `article = models.ManyToManyField(Article)`
- c) `article = models.ForeignKey(  
    Article, on_delete=models.CASCADE  
)`
- d) `article = models.TextField()`

2 CommentSerializer에서 article 필드를 title만 출력하도록 커스터마이징하려면 어떤 구조가 적절한가?

- a) `article = serializers.StringRelatedField()`
- b) `article = ArticleSerializer()`
- c) `article = serializers.PrimaryKeyRelatedField(  
    read_only=True  
)`
- d) `article = 사용자 정의 Serializer(  
    read_only=True  
)`

## Q 각 문제에 올바른 답안을 생각해봅시다

3 read\_only\_fields에 대한 설명으로 올바른 것은?

- a) 유효성 검사에 반드시 포함되어야 한다
- b) 클라이언트 입력값에 따라 값이 저장된다
- c) 응답에는 포함되지 않는다
- d) 생성 및 수정 요청 모두에서 입력을 허용하지 않는다

5 annotate()를 사용하는 목적은 무엇인가?

- a) 모델을 생성하기 위해
- b) 쿼리 결과에 계산된 필드를 추가하기 위해
- c) URL을 라우팅하기 위해
- d) serializer를 초기화하기 위해

4 다음 중 SerializerMethodField에 대한 설명으로 올바른 것은?

- a) 사용자 입력을 받아 저장하는 필드이다
- b) 실제 모델 필드가 반드시 존재해야 한다
- c) 응답용 계산 데이터를 포함할 수 있다
- d) Meta 클래스의 fields에 포함할 수 없다

6 다음 중 Count('comment')에서 'comment'는 무엇을 의미하는가?

- a) 댓글의 모델명
- b) ForeignKey 필드명
- c) 역참조 매니저 이름
- d) serializer의 필드 이름



## Q 각 문제에 올바른 답안을 생각해봅시다

7 get\_object\_or\_404()를 사용하는 이유로 적절한 것은?

- a) 무조건 200 OK 응답을 보장하기 위해
- b) 모델 객체가 없을 때 404로 처리하기 위해
- c) 클라이언트가 보낸 데이터의 유효성을 검사하기 위해
- d) 여러 객체를 한 번에 반환하기 위해

9 다음 중 SerializerMethodField의 특징으로 적절하지 않은 것은?

- a) 읽기 전용 필드이다
- b) get\_<필드명>() 형태의 메서드가 필요하다
- c) annotate 없이도 모델 필드처럼 저장된다
- d) 응답 데이터에 동적으로 생성된 값을 추가할 수 있다

8 get\_list\_or\_404()의 특징으로 옳지 않은 것은?

- a) 조건에 맞는 객체가 없으면 404 응답을 반환한다
- b) 내부적으로 filter()를 사용한다
- c) 항상 빈 리스트를 반환한다
- d) 목록 응답에서도 404 처리를 적용할 수 있다

10 다음 중 읽기 전용 필드를 정의하는 방식으로 옳은 것은?

- a) title = serializers.CharField(read\_only=True)
- b) read\_only\_fields = ('title')
- c) fields = ['id'], read\_only = True
- d) fields = '\_\_all\_\_', required=False



# 확인 문제

## Q 각 문제에 올바른 답안을 생각해봅시다

11 다음 코드 실행 결과로 올바른 응답 형태는?

```
class CommentSerializer(serializers.ModelSerializer):
 class ArticleTitleSerializer(serializers.ModelSerializer):
 class Meta:
 model = Article
 fields = ('title',)

 article = ArticleTitleSerializer(read_only=True)

class Meta:
 model = Comment
 fields = '__all__'
```

- a) "article": { "title": "..."} }
- b) "article": { "id": 3 }
- c) "article": 3
- d) "article": "게시글 제목"

## Q 각 문제에 올바른 답안을 생각해봅시다

- 12 다음 코드에서 유효성 검사 오류가 발생하는 원인은?

```
@api_view(['POST'])
def comment_create(request, article_pk):
 serializer = CommentSerializer(data=request.data)
 if serializer.is_valid(raise_exception=True):
 serializer.save()
 return Response(serializer.data, status=201)
```

- a) article 필드가 read\_only로 되어 있어서
- b) article 값을 클라이언트가 누락했기 때문에
- c) article을 serializer.save()에 전달하지 않았기 때문에
- d) CommentSerializer가 존재하지 않기 때문에

- 13 다음 코드에서 get\_object\_or\_404를 사용하는 목적은?

```
article = get_object_or_404(Article, pk=article_pk)
```

- a) article이 존재하지 않으면 빈 리스트를 반환하기 위해
- b) 존재하지 않는 article에 대한 요청을 404로 응답하기 위해
- c) article이 없을 경우 자동으로 500 오류를 발생시키기 위해
- d) article 모델을 serializer에 자동 등록하기 위해



## Q 각 문제에 올바른 답안을 생각해봅시다

- 14 다음 serializer 정의에 대한 설명으로 옳은 것은?

```
class ArticleSerializer(serializers.ModelSerializer):
 num_of_comments = serializers.SerializerMethodField()

 class Meta:
 model = Article
 fields = '__all__'

 def get_num_of_comments(self, obj):
 return obj.num_of_comments
```

- a) num\_of\_comments는 모델 필드로 저장된다.
- b) annotate() 없이도 항상 응답에 포함된다.
- c) view에서 annotate가 누락되면 오류가 발생할 수 있다.
- d) SerializerMethodField는 클라이언트로부터 입력을 받는다

- 15 다음 코드 실행 결과로 올바른 설명은?

(단, Comment 모델에서 related\_name='comments'로 설정된 경우)

```
Article.objects.annotate(
 num_of_comments=Count('comments')
)
```

- a) 댓글 개수를 계산할 수 없다
- b) annotate된 필드는 serializer에 자동으로 포함된다
- c) Article 모델의 필드가 추가된다
- d) num\_of\_comments 필드는 쿼리 결과에만 존재한다

# 확인 문제



## 정답 및 해설

- 1) c) `article = models.ForeignKey(Article, on_delete=models.CASCADE)`  
 2) d) `article = 사용자 정의 Serializer(read_only=True)` 3) d) 생성 및 수정 요청 모두에서 입력을 허용하지 않는다.  
 4) c) 응답용 계산 데이터를 포함할 수 있다 5) b) 쿼리 결과에 계산된 필드를 추가하기 위해 6) c) 역참조 매니저 이름

1. ForeignKey는 다대일(N:1) 관계를 정의하는 Django 모델 필드입니다. 댓글 여러 개가 하나의 게시글을 참조할 수 있어야 하므로 ForeignKey가 적절합니다. a, b는 각각 1:1, M:N 관계를 나타냅니다.
2. 게시글 전체가 아니라 title만 응답에 포함하려면, article 필드를 사용자 정의 중첩 serializer로 덮어써야 하며, 입력이 아니라 출력만 원하므로 `read_only=True`로 설정해야 합니다.
3. `read_only_fields`에 포함된 필드는 POST나 PUT 요청 시 클라이언트가 값을 제공해도 무시되며, 응답에만 포함됩니다. 생성과 수정 모두에 대해 입력이 금지됩니다.

4. `SerializerMethodField`는 모델에 없는 필드를 응답 결과에 포함시킬 때 사용합니다. 예: 댓글 수, 비율 계산, 이름 합치기 등. 입력용 필드가 아니며, `get_<필드명>()` 메서드로 동작합니다.
5. `annotate()`는 SQL의 집계 함수처럼, 쿼리 결과에 Count, Sum, Avg 등을 통해 임시 필드를 추가합니다. 이를 serializer에서 가공하거나 응답에 포함할 수 있습니다.
6. `Count('comment')`에서 'comment'는 Comment 모델이 `ForeignKey(article=...)`로 Article을 참조할 때 자동 생성되는 역참조 매니저 이름입니다. 기본적으로는 소문자 모델명 기준으로 설정됩니다.



# 확인 문제



## 정답 및 해설

- 7) b) 모델 객체가 없을 때 404로 처리하기 위해 8) c) 항상 빈 리스트를 반환한다 9) c) annotate 없이도 모델 필드처럼 저장된다  
 10) a) title = serializers.CharField(read\_only=True) 11) a) "article": { "title": "..."}  
 12) c) article을 serializer.save()에 전달하지 않았기 때문에

7. get\_object\_or\_404()는 객체가 존재하지 않을 경우 Http404를 발생시킵니다. 이를 통해 API는 500 오류가 아닌, 클라이언트가 이해할 수 있는 404 상태 코드를 반환할 수 있습니다.
8. get\_list\_or\_404()는 filter() 결과가 비어 있을 경우, 빈 리스트가 아닌 404 예외를 발생시킵니다. → 클라이언트는 리소스가 없음을 명확하게 인식할 수 있습니다.
9. SerializerMethodField는 읽기 전용이며, 모델 필드처럼 저장되거나 DB에 존재하는 값이 아닙니다. view나 다른 로직에서 annotate를 통해 값을 제공하거나, 메서드 내부에서 계산해 반환해야 합니다. → annotate 없이 그대로 참조하면 오류가 납니다.

10. read\_only\_fields는 Meta에서 필드명 나열 방식이고, 필드를 명시적으로 정의했다면 직접 read\_only=True를 설정해야 합니다. 특히 커스텀 필드나 중첩 serializer에서 이 방식이 필요합니다.
11. article 필드를 중첩 serializer(ArticleTitleSerializer)로 재정의했기 때문에, 응답 결과에서 article은 숫자가 아닌 JSON 객체로 출력됩니다. 이 객체에는 'title' 필드만 포함되도록 설정했으므로 → { "title": "..."} 형태로 응답됩니다.
12. Comment 모델의 필수 필드인 article은 read\_only\_fields로 지정되어 있어, 클라이언트 요청에서는 입력되지 않습니다. 따라서 view에서 serializer.save(article=article)처럼 명시적으로 값을 넘겨주지 않으면 누락된 필드 오류가 발생합니다.

## 확인 문제



### 정답 및 해설

- 13) b) 존재하지 않는 article에 대한 요청을 404로 응답하기 위해
- 14) c) view에서 annotate가 누락되면 오류가 발생할 수 있다
- 15) d) num\_of\_comments 필드는 쿼리 결과에만 존재한다

13. `get_object_or_404()`는 `get()`처럼 객체를 가져오되, 대상이 없을 경우 `Http404` 예외를 발생시켜 → 클라이언트에게 명확한 "리소스를 찾을 수 없음(404)" 상태를 전달합니다. 따라서 서버 오류(500)가 아닌 정확한 응답 처리를 위한 도구입니다.

14. `num_of_comments = serializers.SerializerMethodField()`는 응답 시 `get_num_of_comments()` 메서드를 호출해 값을 반환합니다. 그 안에서 `obj.num_of_comments`를 참조하는데, 이는 `annotate`로 추가한 필드이기 때문에 → view에서 `annotate(num_of_comments=Count(...))`를 하지 않으면 → 해당 속성이 없다는 `AttributeError`가 발생합니다.

15. `annotate`로 추가된 필드는 모델 정의에는 존재하지 않고, 쿼리 결과에만 임시로 붙는 필드입니다. → 저장되지 않으며, serializer에서 사용하려면 `SerializerMethodField`를 통해 수동으로 포함해야 합니다.



# 활동 정리

## 핵심 키워드

| 개념                                 | 설명                                   | 예시                                                        |
|------------------------------------|--------------------------------------|-----------------------------------------------------------|
| ForeignKey                         | 다른 모델을 참조하여 관계를 설정하는 필드              | <code>article = models.ForeignKey(Article, ...)</code>    |
| 역참조 (related name)                 | 참조된 모델에서 역으로 연결된 데이터를 조회할 수 있게 하는 속성 | <code>article.comment_set, related_name='comments'</code> |
| <code>read_only_fields</code>      | 클라이언트가 수정할 수 없는 필드, 응답에만 포함됨         | <code>read_only_fields = ('article',)</code>              |
| <code>SerializerMethodField</code> | 메서드를 통해 응답용 필드를 생성하는 읽기 전용 필드        | <code>num_of_comments = SerializerMethodField()</code>    |
| <code>annotate</code>              | 쿼리 단계에서 집계 계산 필드를 동적으로 생성하는 ORM 함수   | <code>annotate(num_of_comments=Count('comment'))</code>   |
| <code>get_object_or_404</code>     | 객체가 없을 경우 404 예외를 발생시키는 함수           | <code>get_object_or_404(Article, pk=1)</code>             |
| <code>get_list_or_404</code>       | 필터 결과가 비어 있으면 404 예외를 발생시키는 함수       | <code>get_list_or_404(Comment, article=article)</code>    |

<표2\_Django\_DRF 2\_핵심 키워드>

# 활동 정리

## 오늘 공부한 내용 요약 및 정리

- 댓글과 게시글 사이의 N:1 관계는 ForeignKey로 정의되며, Django에서는 기본적으로 참조된 객체의 id만 응답에 포함합니다.
- 응답을 사람이 이해하기 쉬운 구조로 바꾸기 위해, 중첩 serializer를 활용해 title과 같은 필드를 직접 추출해 표현할 수 있습니다.
- 읽기 전용 필드(read\_only\_fields)는 클라이언트가 수정할 수 없는 값을 응답에만 포함시킬 때 사용하며, view 함수에서 .save(article=...)처럼 직접 할당이 필요합니다.
- 쿼리 시점에서 annotate를 사용하면, 예를 들어 댓글 수처럼 계산된 필드를 SerializerMethodField를 통해 응답에 포함할 수 있습니다.
- 객체나 목록이 존재하지 않을 경우 적절한 404 응답을 반환하도록 get\_object\_or\_404, get\_list\_or\_404를 활용해 API 응답의 명확성을 유지할 수 있습니다.
- View에서는 데이터를 준비하고, Serializer는 이를 가공하고 표현합니다.  
→ 복잡한 집계와 쿼리 조작은 View에서, 표현 형식 제어는 Serializer에서 담당합니다.



## 활동 정리

**배달 앱에서 여러 사용자가 여러 음식점을 즐겨찾기 등록한다고 생각해 봅시다!**

• 예를 들어, 사용자 A는 치킨집과 피자집을, 사용자 B는 피자집과 분식집을 등록할 수 있어요.

1. 사용자와 음식점은 M:N(다대다) 관계를 가집니다.
2. UserFavoriteStore와 같은 {{ 중개 모델 }}을 만들어 관계를 정의하고,
3. DRF를 통해 즐겨찾기 목록 조회, 새 즐겨찾기 추가 기능을 구현합니다.
4. {{ read\_only\_fields }}를 활용해, 클라이언트가 직접 바꾸면 안 되는 값(예: 등록 시간)을 보호할 수 있어요.
5. {{ SerializerMethodField }}로 가공된 데이터(예: 총 즐겨찾기 수)를 함께 제공할 수도 있습니다

- DRF는 모델 간 복잡한 관계도 쉽게 표현할 수 있게 도와줍니다.
- 중간 테이블을 통한 M:N 구조도 깔끔하게 직렬화할 수 있고,
- 읽기 전용 필드와 커스텀 필드를 통해 API의 안정성과 표현력을 높일 수 있습니다

# 감사합니다

